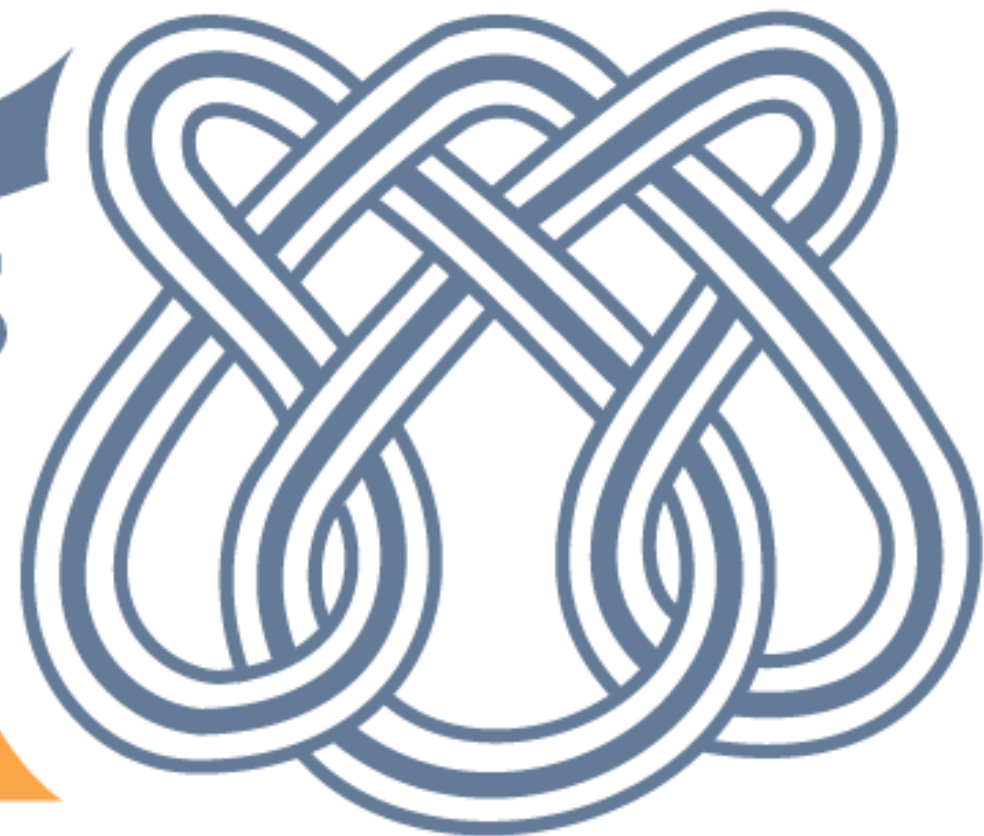


ICMC USP
SÃO CARLOS

SSC502 – Laboratório de ICC I

Ponteiros

Prof.Dr. Danilo Spatti



- Variável:
 - É um espaço reservado de memória usado para guardar um valor que pode ser modificado pelo programa.

- Ponteiro:
 - É um espaço reservado de memória usado para guardar o endereço de memória de uma outra variável.
 - Um ponteiro é uma variável como qualquer outra do programa – sua diferença é que ela não armazena um valor inteiro, real, caractere ou booleano.
 - Ela serve para armazenar endereços de memória (são valores inteiros sem sinal).

- É o asterisco (*) que **informa** ao **compilador** que aquela variável **não vai** guardar um **valor** mas sim um **endereço** para o tipo especificado.

```
int *x;  
float *y;  
struct *p;
```

- Na linguagem C, quando **declaramos** um **ponteiro** nós informamos ao compilador para que **tipo** de **variável** vamos apontá-lo.
- Um ponteiro **int*** aponta para um **inteiro**, isto é, **int**.
- Esse **ponteiro** guarda o **endereço** de **memória** onde se **encontra armazenada** uma **variável** do tipo **int**.

- Ponteiros **apontam** para uma **posição** de **memória**.
- Cuidado: Ponteiros **não inicializados** apontam para um lugar **indefinido**.
- Exemplo `int *p;`

Memória		
posição	variável	conteúdo
119		
120	<code>int *p</code>	????
121		
122		
123		

- Os ponteiros devem ser **inicializados antes de serem usados**.
- Assim, devemos **apontar** um **ponteiro** para um **lugar conhecido**.
- Podemos **apontá-lo** para uma **variável** que **já exista** no **programa**.

Memória		
posição	variável	conteúdo
119		
120	int *p	122
121		
122	int c	10
123		



- O ponteiro **armazena** o endereço **da** variável para **onde** ele **aponta**.
- Para saber o **endereço** de **memória** de uma **variável** do nosso **programa**, usamos o **operador &**.
- Ao armazenar o **endereço**, o **ponteiro** estará **apontando** para aquela **variável**.

```
1  # include <stdio.h>
2
3  int main()
4  = {
5      int c = 10;
6      int *p;
7      p = &c;
8      printf("c = %d\n", c);
9      printf("p = %d\n", p);
10     return 0;
11 }
```

```
c = 10
p = 6356744
```

```
Process returned 0 (0x0)   execution time : 0.016 s
Press any key to continue.
```


- Tendo um ponteiro armazenado um endereço de memória, como saber o valor guardado dentro dessa posição?
- Para acessar o valor guardado dentro de uma posição na memória apontada por um ponteiro, basta usar o operador asterisco “*” na frente do nome do ponteiro.

- De modo **geral**, um **ponteiro** só pode **receber** o **endereço** de **memória** de uma **variável** do mesmo **tipo** do **ponteiro**.
- Isso ocorre porque **diferentes** tipos de **variáveis** ocupam **espaços** de **memória** de tamanhos **diferentes**.
- Na verdade, nós **podemos atribuir** a um ponteiro de **inteiro** (`int *`) o endereço de uma variável do **tipo float**. No entanto, o **compilador assume** que qualquer **endereço** que esse ponteiro **armazene obrigatoriamente** apontará para uma variável do tipo **int**.
- Isso gera problemas na interpretação dos valores.

```
1  # include <stdio.h>
2  # include <locale.h>
3
4  int main()
5  = {
6      setlocale(LC_ALL, "Portuguese");
7      int c = 10;
8      int *p;
9      p = &c;
10     printf("Conteúdo de c: %d\n", c);
11     printf("Conteúdo apontado por p: %d\n", *p);
12     *p = 12;
13     printf("Novo conteúdo apontado por p: %d\n", *p);
14     printf("Conteúdo de c: %d\n", c);
15     return 0;
16 }
```

Conteúdo de c: 10
Conteúdo apontado por p: 10
Novo conteúdo apontado por p: 12
Conteúdo de c: 12

Process returned 0 (0x0) execution time : 0.016 s
Press any key to continue.

```
1 # include <stdio.h>
2
3 int main()
4 {
5     int *p, *p1, x = 10;
6     float y = 20.0;
7     p = &x;
8     printf("Conteudo apontado por p: %d\n", *p);
9     p1 = p;
10    printf("Conteudo apontado por p1: %d\n", *p1);
11    p = &y;
12    printf("Conteudo apontado por p: %d\n", *p);
13    printf("Conteudo apontado por p: %f\n", *p);
14    printf("Conteudo apontado por p: %f\n", *((float*)p));
15    return 0;
16 }
```

```
Conteudo apontado por p: 10
Conteudo apontado por p1: 10
Conteudo apontado por p: 1101004800
Conteudo apontado por p: 0.000000
Conteudo apontado por p: 20.000000
```

```
Process returned 0 (0x0)   execution time : 0.016 s
Press any key to continue.
```

- Apenas **duas operações aritméticas** podem ser utilizadas com endereço armazenado pelo ponteiro: **adição** e **subtração**.
- Podemos apenas somar e subtrair valores INTEIROS
 - $p++$: soma +1 no endereço armazenado no ponteiro.
 - $p--$: subtrai 1 no endereço armazenado no ponteiro.
 - $p = p+15$: soma +15 no endereço armazenado no ponteiro.

- As operações de **adição** e **subtração** no endereço **dependem** do **tipo** de dado que o ponteiro **aponta**.
- O tipo **int ocupa** um espaço de **4 bytes** na memória.
- Assim, nas operações de **adição** e **subtração** são adicionados/subtraídos **4 bytes** por incremento/decremento.

Memória		
posição	variável	conteúdo
119		
120	int a	10
121		
122		
123		
124	int b	20
125		
126		
127		
128	char c	'k'
129	char d	's'
130		

- Existem **operações** que são **ilegais** com **ponteiros**.
- Dividir ou multiplicar ponteiros.
- Somar o endereço de dois ponteiros.
- Não se pode adicionar ou subtrair valores dos tipos float ou double de ponteiros.

- Sobre o **conteúdo** apontado pelo ponteiro, todas as **operações** são **válidas**.
- $(*p) ++$: **incrementa** o **conteúdo** apontado pelo ponteiro
- Multiplicar o conteúdo da variável apontada pelo ponteiro por 10:
 - $*p = (*p) * 10;$

- `==` e `!=` para saber se dois ponteiros são iguais ou diferentes.
- `>`, `<`, `>=`, `<=` para saber qual ponteiro aponta uma posição mais alta na memória.
- Exemplo: código para saber se dois ponteiros apontam para o mesmo endereço.

```
1 # include <stdio.h>
2
3 int main()
4 {
5     int *p, *q, x, y;
6     p = &x;
7     q = &y;
8     if (p == q) printf("Ponteiros iguais\n");
9     else printf("Ponteiros diferentes\n");
10    return 0;
11 }
```

- Normalmente, um **ponteiro** aponta para um **tipo específico** de dado.
- Um ponteiro **genérico** é um ponteiro que **pode apontar** para **qualquer** tipo de dado.
- Declaração
`void *nome_ponteiro;`

```
1 # include <stdio.h>
2
3 int main()
4 {
5     void *pp;
6     int *p1, p2 = 10;
7     p1 = &p2;
8     pp = &p2;
9     printf("Endereco em pp: %p\n", pp);
10    pp = &p1;
11    printf("Endereco em pp: %p\n", pp);
12    pp = p1;
13    printf("Endereco em pp: %p\n", pp);
14    return 0;
15 }
```

```
Endereco em pp: 0060FF04
Endereco em pp: 0060FF08
Endereco em pp: 0060FF04
```

```
Process returned 0 (0x0)   execution time : 0.016 s
Press any key to continue.
```

- Para acessar o **conteúdo** de um ponteiro **genérico** é preciso antes **convertê-lo** para o **tipo** de **ponteiro** com o qual se deseja trabalhar.
- Isso é feito via `type cast`

```
1  # include <stdio.h>
2
3  int main()
4  = {
5      void *pp;
6      int p2 = 10;
7      pp = &p2;
8      printf("Conteudo: %d\n", *(int*)pp);
9      return 0;
10 }
```

Conteudo: 10

Process returned 0 (0x0) execution time : 0.016 s
Press any key to continue.

- Vetores são **agrupamentos** de **dados** do **mesmo tipo** na memória, alocados **contiguamente**.
- Quando **declaramos** um vetor, informamos ao **computador** para **reservar** uma certa **quantidade** de **memória** a fim de armazenar os elementos do vetor de forma **sequencial**.
- O **computador** nos **devolve** um **ponteiro** que **aponta** para o **começo** dessa sequência de bytes na memória.

- O nome do vetor (sem índice) é apenas um **ponteiro** que **aponta** para o **primeiro elemento** do **vetor**.

```
int vet[5] = {1, 2, 3, 4, 5};  
int *p;  
p = vet;
```

Memória		
posição	variável	conteúdo
119		
120		
121	int *p	123
122		
123	int vet[5]	1
124		2
125		3
126		4
127		5
128		

- Os colchetes `[ ]` substituem o uso conjunto de operações aritméticas e de acesso ao conteúdo (operador “`*`”) no acesso ao conteúdo de uma posição de um vetor ou ponteiro.
- O valor entre colchetes é o deslocamento a partir da posição inicial do vetor.
- Nesse caso, `p[2]` equivale a `* (p+2)`.

```
1 # include <stdio.h>
2
3 int main()
4 {
5     int vet[5] = {1,2,3,4,5};
6     int *p;
7     p = vet;
8     printf("Terceiro elemento de vet: %d ou %d",p[2],*(p+2));
9     return 0;
10 }
```

```
Terceiro elemento de vet: 3 ou 3
Process returned 0 (0x0)   execution time : 0.016 s
Press any key to continue.
```

```
int vet[5] = {1, 2, 3, 4, 5};  
int *p;  
p = vet;
```

- Neste exemplo:
 - `*p` é equivalente a `vet[0]`;
 - `vet[índice]` é equivalente a `*(p+índice)`;
 - `vet` é equivalente a `&vet[0]`;
 - `&vet[índice]` é equivalente a `(vet+índice)`;

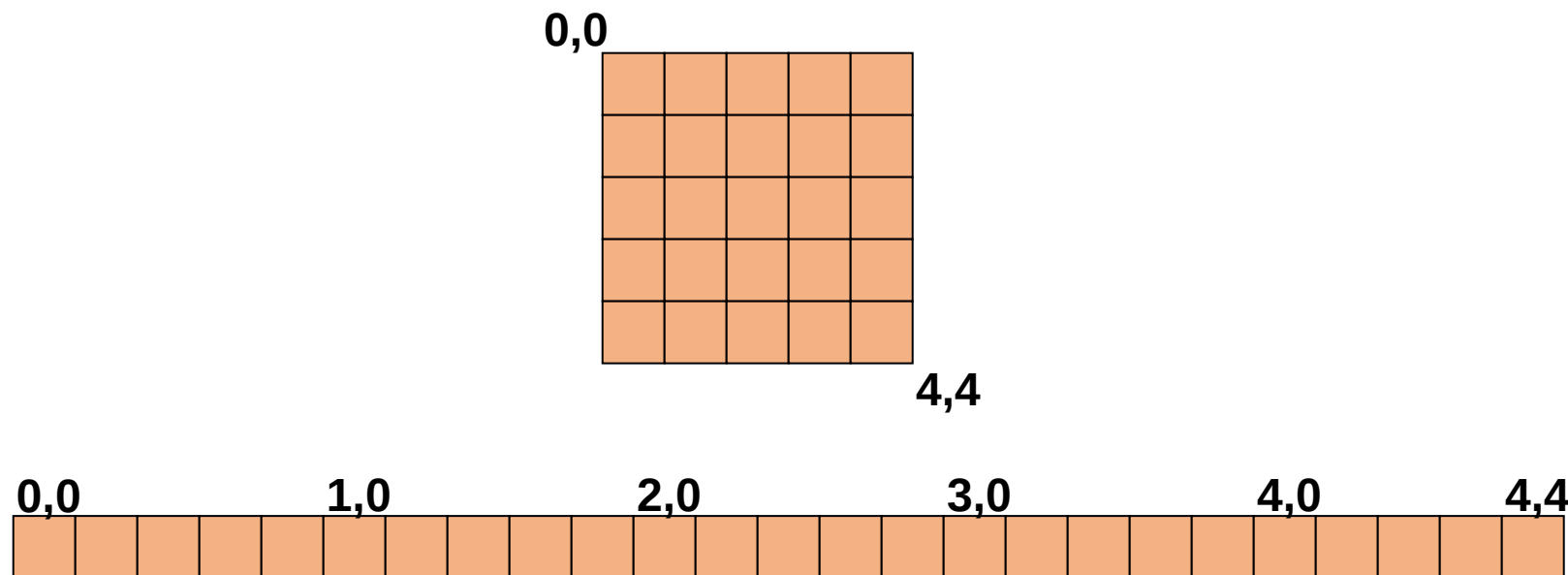
Usando vetor

```
int main() {  
    int vet[5] = {1, 2, 3, 4, 5};  
    int *p = vet;  
    int i;  
    for (i = 0; i < 5; i++)  
        printf("%d\n", p[i]);  
  
    return 0;  
}
```

Usando ponteiro

```
int main() {  
    int vet[5] = {1, 2, 3, 4, 5};  
    int *p = vet;  
    int i;  
    for (i = 0; i < 5; i++)  
        printf("%d\n", *(p+i));  
  
    return 0;  
}
```

- Apesar de **terem** mais de uma **dimensão**, na **memória** os dados são **armazenados linearmente**.
- Exemplo: `int mat[5][5];`



Usando matriz

```
int main() {  
    int mat[2][2] = {{1,2},{3,4}};  
    int i,j;  
    for(i=0;i<2;i++)  
        for(j=0;j<2;j++)  
            printf("%d\n", mat[i][j]);  
  
    return 0;  
}
```

Usando ponteiro

```
int main() {  
    int mat[2][2] = {{1,2},{3,4}};  
    int *p = &mat[0][0];  
    int i;  
    for(i=0;i<4;i++)  
        printf("%d\n", *(p+i));  
  
    return 0;  
}
```

- A linguagem C permite criar **ponteiros** com **diferentes níveis** de apontamento.
- É possível criar um **ponteiro** que **aponte** para outro **ponteiro**, criando assim **vários** níveis de **apontamento**.
- Assim, um **ponteiro** poderá apontar para outro **ponteiro**, que, por sua vez, aponta para outro **ponteiro**, que aponta para um terceiro **ponteiro** e assim por **diante**.

- Um **ponteiro** para um **ponteiro** é como se você **anotasse** o **endereço** de um papel que tem o **endereço** da casa do seu amigo.
- Podemos declarar um ponteiro para um ponteiro com a seguinte notação
`tipo_ponteiro **nome_ponteiro;`
- Acesso ao conteúdo
 - `**nome_ponteiro` é o conteúdo final da variável apontada;
 - `*nome_ponteiro` é o conteúdo do ponteiro intermediário.


```
int x = 10;
Int *p1 = &x;
Int **p2 = &p1;
//endereço em p2
printf("Endereco em p2: %p\n", p2);
//Conteúdo do endereço
printf("Conteúdo em *p2: %p\n", *p2);
//Conteúdo do endereço do endereço
printf("Conteúdo em **p2: %d\n", **p2);
```

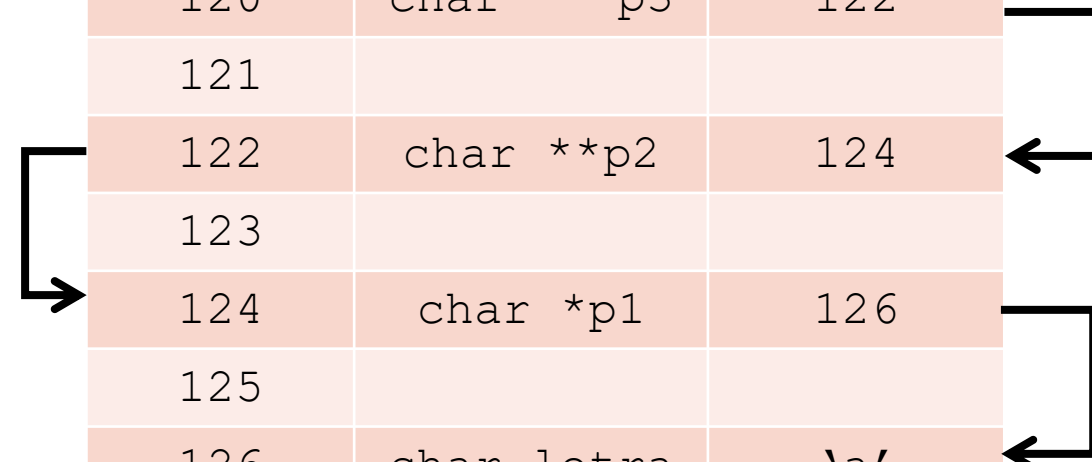
Memória		
posição	variável	conteúdo
119		
120		
121		
122	int **p2	124
123		
124	int *p1	126
125		
126	int x	10
127		

- É a **quantidade** de asteriscos (*) na declaração do ponteiro que indica o **número** de **níveis** de **apontamento** que ele possui.

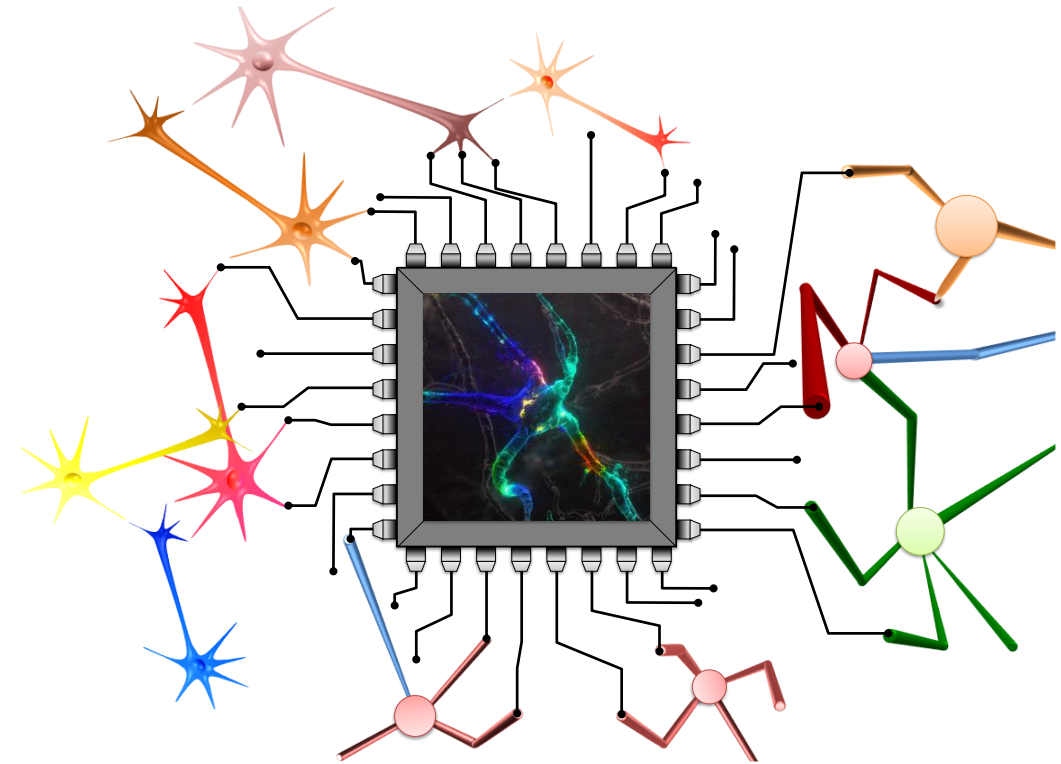
```
int x; // Variável Inteira  
int *p1; // Ponteiro 1 nível  
int **p2; // Ponteiro 2 níveis  
int ***p3; // Ponteiro 3 níveis
```

```
1 # include <stdio.h>
2
3 int main()
4 {
5     char letra = 'a';
6     char *p1;
7     char **p2;
8     char ***p3;
9     p1 = &letra;
10    p2 = &p1;
11    p3 = &p2;
12    printf("Conteudo p1: %d\n", p1);
13    printf("Conteudo p2: %d\n", p2);
14    printf("Conteudo p3: %d\n", p3);
15    return 0;
16 }
```

Memória		
posição	variável	conteúdo
119		
120	char ***p3	122
121		
122	char **p2	124
123		
124	char *p1	126
125		
126	char letra	'a'
127		



spatti@icmc.usp.br



GE4Bio – Grupo de Estudos em Sinais Biológicos